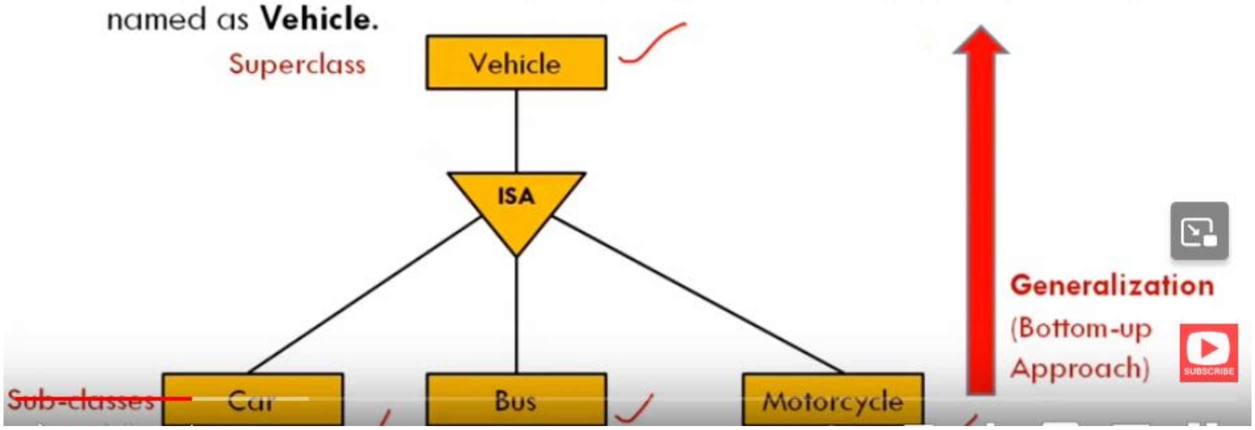
Extended Entity Relationship (ER) Features

- As the complexity of data increased in the late 1980s, it became more and more difficult to use the traditional ER Model for database modelling. Hence some improvements or enhancements were made to the existing ER Model to make it able to handle the complex applications better.
- Hence, as part of the **Extended ER Model**, along with other improvements, three new concepts were added to the existing ER Model:
  1. **Generalization**
  2. **Specialization**
  3. **Aggregation**

- **Generalization** is the process of extracting common properties from a set of entities and create a generalized entity from it.
- **Generalization is a "bottom-up approach"** in which two or more entities can be combined to form a higher level entity if they have some attributes in common.
  - subclasses are combined to make a superclass.
- **Generalization is used** to emphasize the similarities among lower-level entity set and to hide differences in the schema

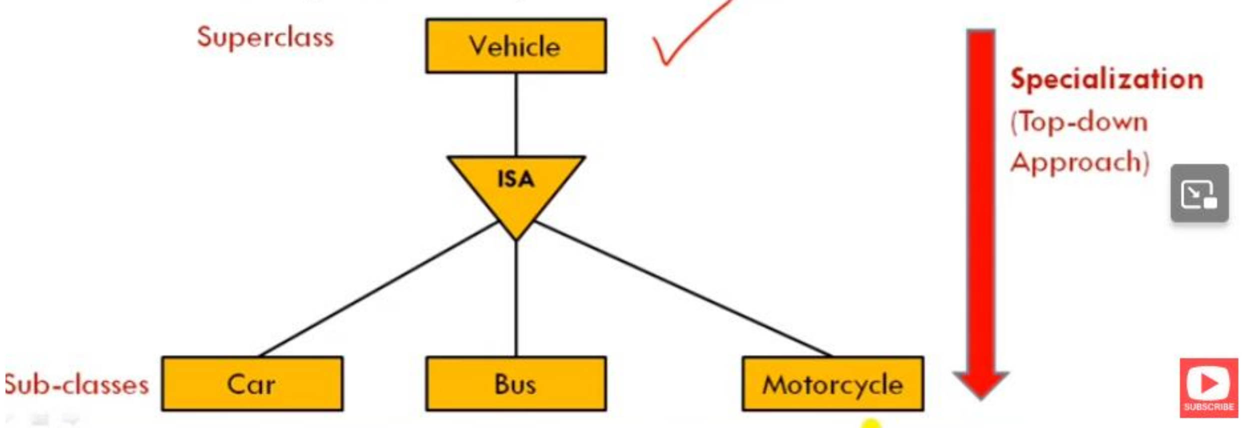
- Consider we have 3 sub entities Car, Bus and Motorcycle. Now these three entities can be generalized into one higher-level entity (or super class) named as **Vehicle.**

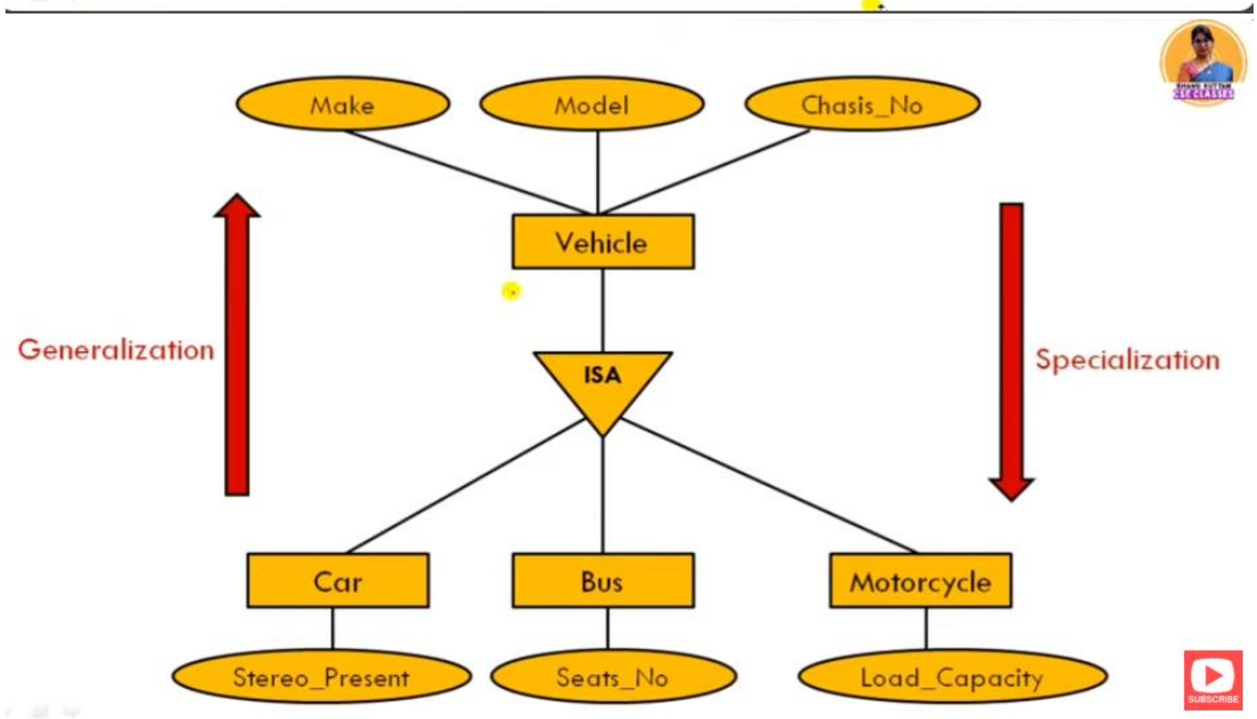


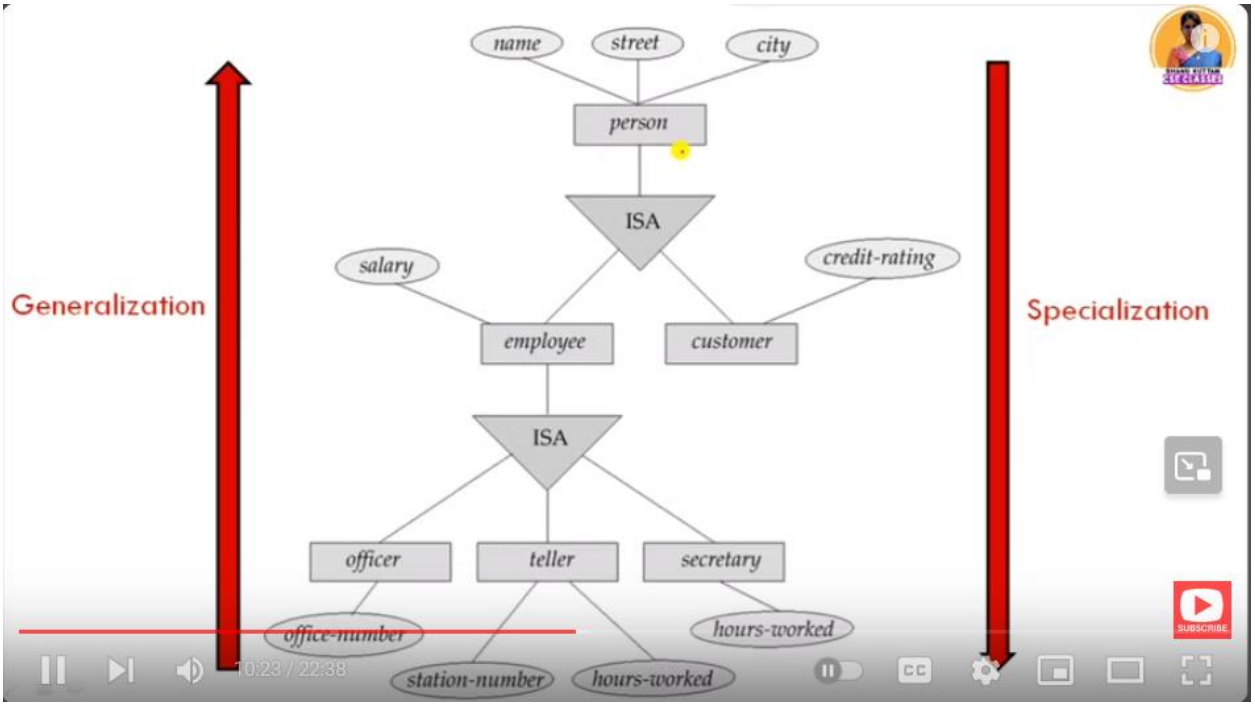
Specialization

- Specialization is **opposite** of Generalization
- In **Specialization**, **an entity is broken down into sub-entities based on their characteristics.**
- **Specialization is a "Top-down approach"** where higher level entity is specialized into two or more lower level entities.
- **Specialization is used** to identify the subset of an entity set that shares some distinguishing characteristics.
- **Specialization** can be repeatedly applied to refine the design of schema
- depicted by triangle component labeled **ISA**

- **Vehicle** entity can be a Car, Truck or Motorcycle.
  - Normally, the superclass is defined first, the subclass and its related attributes are defined next, and relationship set are then added.

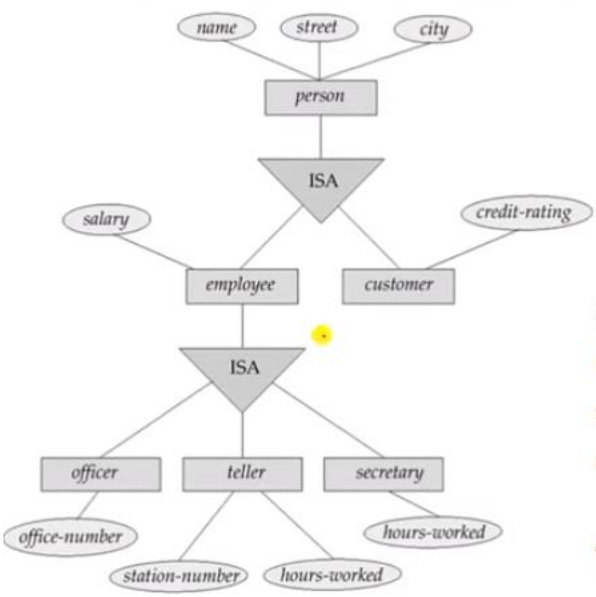






S

How Schema or Tables can be formed?



Four tables can be formed:

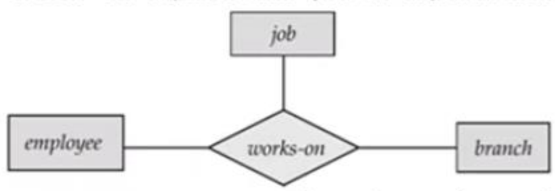
1. **customer** (name, street, city, credit_rating)
2. **officer** (name, street, city, salary, office_number)
3. **teller** (name, street, city, salary, station_number, hours_worked)
4. **secretary** (name, street, city, salary, hours_worked)

Aggregation

- **Aggregation** is used when we need to express a **relationship among relationships**
- **Aggregation** is an **abstraction** through which **relationships are treated as higher level entities**
- **Aggregation** is a process when a **relationship between two entities is considered as a single entity** and again this single entity has a **relationship** with **another entity**

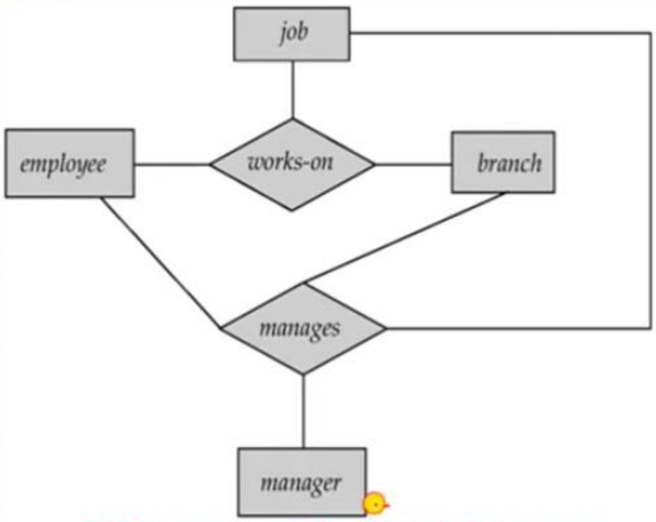
Example: (Relationship of Relations)

- Basic E-R model can't represent relationships involving other relationships
- Consider a ternary relationship **works_on** between **Employee, Branch and Job.**
  - Am **employee** ***works on*** a particular **job** at a particular **branch**



- Suppose we want to assign a **manager** for jobs performed by an employee at a branch (i.e. want to assign managers to each employee, job, branch combination)
- Need a separate manager entity-set
- Relationship between each manager, employee, branch, and job entity

Example: (Redundant Relationship)

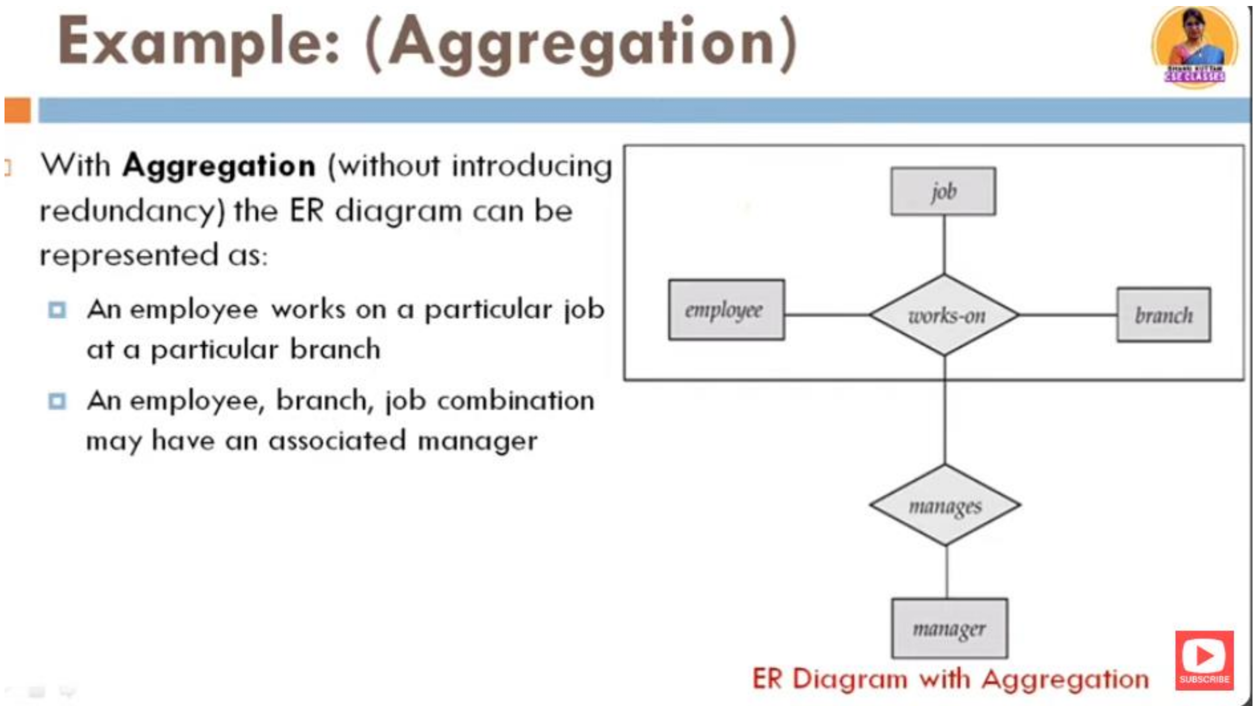


ER Diagram with Redundant Relationship

- Relationship sets ***works-on*** and ***manages*** represent overlapping (redundant) information
  - Every *manages* relationship corresponds to a *works-on* relationshi
  - However, some *works-on* relationships may not correspond to any *manages* relationships
    - So we can't discard the *works-o* relationship

Aggregation

- Eliminate this redundancy via ***aggregation***
  - Treat relationship as an abstract entity
  - Allows relationships between relationships
  - Abstraction of relationship into new entity



https://www.youtube.com/watch?v=1G4agXWNr_M&t=10s